DESARROLLO BASADO EN COMPONENTES VS. DESARROLLO BASADO EN SERVICIOS

Comparación aplicada al proyecto ÓrbiKa — marketplace de abarrotes con stock controlado

.Brant Antony Chata Choque

· Renzo Fernando Loyola Vilca

Agenda

1. Marco teórico: DBC vs. DBS
2. Contexto del proyecto ÓrbiKa
3. Selección y composición de componentes
4. Interfaces, contratos y conexiones
5. Arquitectura de servicios (DBS)
6. Aseguramiento de calidad: casos de prueba
7. Comparación aplicada, conclusiones y recomendaciones

DBC vs. DBS: dos formas de organizar el software

Desarrollo Basado en Componentes (DBC)

Divide el sistema en unidades de software reutilizables (componentes) que exponen sus funciones mediante interfaces bien definidas, se ensamblan dentro de un mismo proceso/aplicación y se comunican mediante llamadas directas de código.

Énfasis en: reutilización interna, bajo acoplamiento vía interfaces, composición dentro de una misma base de código.

Desarrollo Basado en Servicios (DBS)

Organiza el sistema como un conjunto de servicios independientes, expuestos por contratos de red (APIs/REST) y consumidos por distintos clientes a través de solicitudes remotas (HTTP/JSON), pudiendo desplegarse y escalar por separado.

Énfasis en: independencia de despliegue, comunicación remota estandarizada, consumo por múltiples clientes (web, futuras apps).

ÓrbiKa: requisitos funcionales base

Código	Requisito funcional	Rol en la comparación
RF-01	Registro y autenticación diferenciada (comprador / vendedor)	Base para DBS (Supabase Auth, servicio externo)
RF-02	Autorización por rol en rutas protegidas	Expuesta vía componente y vía API
RF-07	Compra con stock consistente (control transaccional)	Caso central: componente + servicio + función SQL

El mismo sistema ÓrbiKa se analiza primero como composición de componentes internos, y luego como arquitectura de servicios consumidos por el cliente React.

Componentes identificados en ÓrbiKa

Código	Componente	Responsabilidad
C01	Presentación React	Páginas y componentes de interfaz
C02	AuthContext	Sesión y perfil del usuario en el cliente
C03	Gestión de productos	Publicar, listar, editar productos
C04	Gestión de pedidos	Crear, listar, cancelar, calificar pedidos
C05	Autenticación y autorización	Validación de rol y permisos
C06	Gestión de imágenes	Carga y asociación de fotos de producto
C07	Acceso a datos	Cliente Supabase / funciones RPC transaccionales

Por qué estos componentes

Cohesión

Cada componente agrupa una sola responsabilidad de negocio (productos, pedidos, imágenes).

Reutilización

AuthContext y Acceso a datos son usados por todos los demás componentes sin duplicar lógica.

Punto único de escritura

C07 es el único autorizado a modificar productos y pedidos, evitando inconsistencias de stock.

Aislamiento de reglas

C05 centraliza la autorización por rol, independiente de la vista que la invoque.

Composición por capas

Presentación

C01 Presentación React · C02 AuthContext

Lógica de aplicación

C03 Productos · C04 Pedidos · C05 Autorización · C06 Imágenes

Persistencia

C07 Acceso a datos → Supabase / PostgreSQL (tablas + funciones transaccionales)

Flujo: registro e inicio de sesión

1. El usuario ingresa datos en el formulario (C01); si elige rol vendedor, se exigen negocio y ubicación.
2. C02 (AuthContext) invoca a Supabase Auth con las credenciales y recibe la sesión.
3. C05 valida el rol asociado al perfil antes de habilitar rutas protegidas.
4. C07 persiste/consulta el perfil en la base de datos gestionada por Supabase.

Flujo: compra con stock consistente

1. El comprador define cantidad válida y se genera una clave de idempotencia (crypto.randomUUID) en C01.
2. C04 envía la solicitud de compra hacia C07 (acceso a datos).
3. C07 ejecuta crear_pedido_transaccion: bloquea la fila del producto (FOR UPDATE), serializa por clave de idempotencia (advisory lock) y valida stock, vencimiento, categoría y estado del vendedor.
4. Si todo es válido, se crea el pedido, se descuenta el stock y se responde de forma atómica; una confirmación repetida no duplica el efecto.

Interfaces expuestas por los componentes

Interfaz	Expuesta por	Operaciones principales
IProductos	C03	listar, obtener, publicar, editar
IPedidos	C04	crear, listar, cancelar, calificar
IAutorizacion	C05	validarRol, requiereSesion
IImagenes	C06	subirFoto, asociarAProducto
IRepositorio	C07	consultar, ejecutarRPC (acceso único a datos)

Contratos funcionales entre componentes

Operación	Entrada	Salida / garantía
crearPedido	productoId, cantidad, claveIdempotencia	Pedido creado y stock reducido exactamente una vez
publicarProducto	datos del producto (rol vendedor)	Producto visible en el marketplace
actualizarEstadoPedido	pedidoId, nuevoEstado	Transición de estado validada por rol
calificarPedido	pedidoId, puntuación	Calificación asociada al pedido entregado

De componentes a servicios

Código	Servicio	Naturaleza
SW-01	Servicio de productos	Propio — Express (/api/productos)
SW-02	Servicio de pedidos	Propio — Express (/api/pedidos)
SW-03	Servicio de imágenes	Propio — Express (/api/imagenes)
SW-04	Autenticación (Supabase Auth)	Externo gestionado — consumido directo desde el cliente

SW-04 es un servicio gestionado por Supabase: ÓrbiKa no implementa su lógica, solo lo consume desde React (RF-01, RF-02, RF-13).

Endpoints reales expuestos por la API

Servicio	Método y ruta	Función
SW-01	GET / POST /api/productos	Listar y publicar productos
SW-01	PUT /api/productos/:id	Editar producto (rol vendedor)
SW-02	POST /api/pedidos	Crear pedido (ejecuta crear_pedido_transaccion)
SW-02	GET /api/pedidos	Listar pedidos del usuario
SW-02	PATCH /api/pedidos/:id/cancelar	Cancelar pedido
SW-02	PATCH /api/pedidos/:id/estado	Actualizar estado del pedido
SW-02	POST /api/pedidos/:id/calificacion	Calificar pedido entregado
SW-03	POST /api/imagenes	Subir y asociar imagen a producto

Consumo desde React y puntos clave de operación

Consumo desde el cliente

El cliente React usa axios con el token JWT de Supabase para los servicios propios (SW-01/02/03), y supabase-js directo (anon key) para autenticación (SW-04).

Acceso a datos

El backend usa la service role key de Supabase solo en el servidor; todas las escrituras críticas (pedidos) pasan por funciones RPC transaccionales, nunca por SQL directo desde el cliente.

Errores y respuestas

Códigos reales usados: 400 validación, 401 sin sesión, 403 rol incorrecto, 404 no encontrado, 409 conflicto de estado, 502 error del proveedor (Supabase).

Mantenimiento

Migraciones versionadas en supabase/migrations/; pruebas automatizadas (Vitest) cubren servicios, validaciones y la transacción SQL mediante PGlite.

Evidencia real de pruebas ejecutadas

91/91
Pruebas backend (Vitest) pasando

49/49
Pruebas frontend (Vitest) pasando

7/10
Casos de prueba CP-01..CP-10 con evidencia PASS

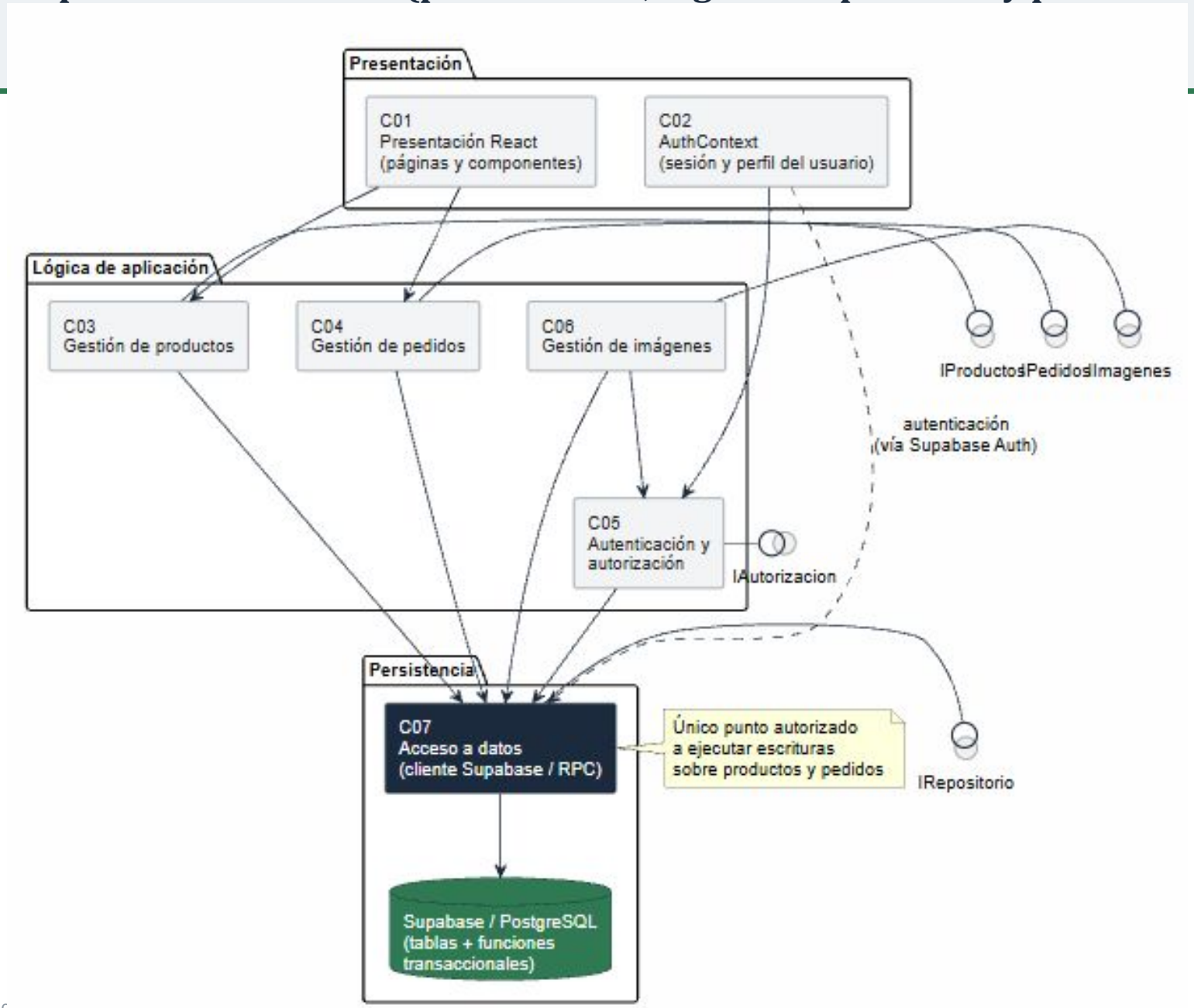
3/10
Casos pendientes de ejecución (concurrencia / límites)

Caso	RF	Descripción	Resultado
CP-01	RF-01	Contraseña débil o sin coincidencia es rechazada	PASS
CP-07	RF-07	Compra dentro de stock crea pedido y reduce stock una vez	PASS
CP-08	RF-07	Confirmación repetida (idempotencia) no duplica el pedido	PASS
CP-09	RF-07	Compra que excede el stock disponible	PENDIENTE
CP-10	RF-07	Compras concurrentes sobre el mismo stock	PENDIENTE

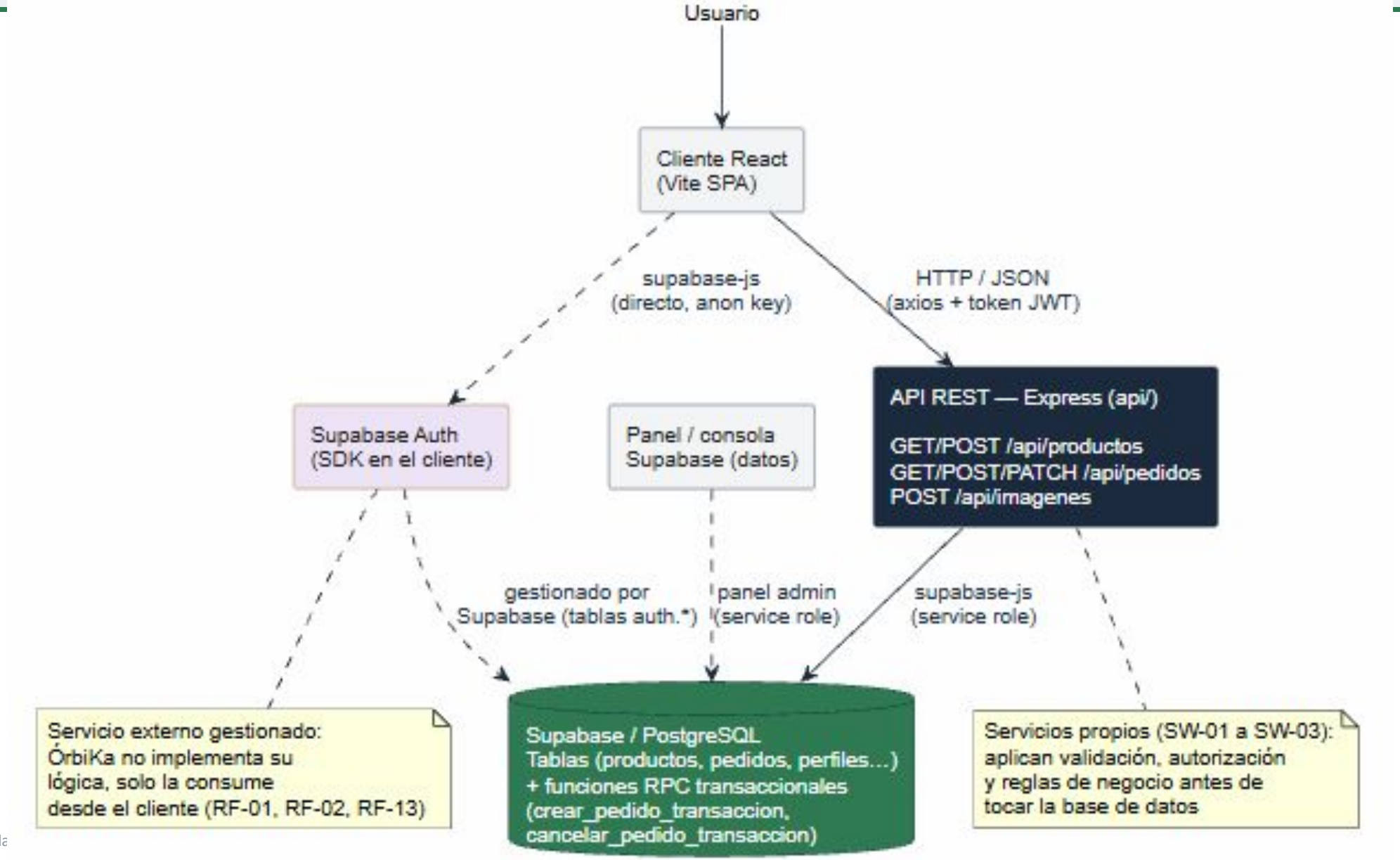
DBC vs. DBS en ÓrbiKa, lado a lado

Criterio	DBC (componentes)	DBS (servicios)
Unidad principal	Componentes internos (C01-C07)	Servicios HTTP (SW-01 a SW-04)
Comunicación	Llamadas de función en el mismo proceso	HTTP/JSON entre cliente y servidor
Ejemplo real	C04 invoca a C07 directamente	React llama a POST /api/pedidos
Reutilización	Dentro de la misma aplicación	Por cualquier cliente que consuma la API
Fallos principales	Acoplamiento si se ignoran interfaces	Latencia y disponibilidad de red
Aplicación conjunta	Compone la lógica interna del backend	Expone esa lógica a React y futuros clientes

Composición de componentes de ÓrbiKa (presentación, lógica de aplicación y persistencia).



Arquitectura de servicios de ÓrbiKa: API REST propia (Express) y servicio gestionado de autenticación (Supabase Auth), ambos sobre la misma base PostgreSQL.



Conclusiones

- ÓrbiKa combina ambos enfoques: DBC organiza la lógica interna del backend, y DBS la expone como API consumida por el cliente React.
- El caso RF-07 (compra con stock consistente) demuestra que la robustez no depende del estilo arquitectónico en sí, sino del diseño de la transacción (FOR UPDATE + advisory lock) en C07/SW-02.
- La autenticación (SW-04) evidencia que no todo servicio necesita ser construido: un servicio externo bien integrado reduce superficie de mantenimiento propio.
- El 70% de los casos de prueba definidos cuentan con evidencia real de ejecución; el resto requiere pruebas de carga y límites antes del cierre del sprint.